

# QRPay SDK - Technical Handoff & Overview

## 1. Overview

This document outlines the architecture, features, and test results for the initial v0.1.0 release of the QRPay SDK. Our primary goal was to build a native-feeling, cross-platform Flutter scanner optimized specifically for payment QR codes (like EMVCo and UPI). We focused heavily on keeping the footprint small and the camera boot-up times minimal, addressing the common pain points of generic Flutter camera plugins.

## 2. Architecture Decisions

We decided to use a federated plugin structure. This allowed us to keep the Dart API clean while directly interfacing with Android's CameraX and iOS's AVFoundation natively.

- `qrpay_sdk`: The app-facing package (Dart).
- `qrpay_sdk_platform_interface`: The MethodChannel/EventChannel contracts.
- `qrpay_sdk_android`: Native Android implementation using Google's ML Kit.
- `qrpay_sdk_ios`: Native iOS implementation using Apple's Vision framework.

Using EventChannels allowed us to stream camera frames back to Dart without the overhead of repeated MethodChannel calls, which kept the UI thread unblocked.

## 3. Performance & Speed Optimizations

To ensure the scanner feels faster than typical payment apps, we applied a few key native optimizations:

- Resolution & Format: We forced the camera output to 720p (1280x720) in YUV\_420\_888 format. This is enough resolution to reliably detect a QR code from a distance without bottlenecking the CPU/GPU like a 1080p frame would.
- Backpressure Strategy: On Android, we set the analyzer to STRATEGY\_KEEP\_ONLY\_LATEST. Instead of queuing frames, if the ML engine is busy, the camera drops the frame. This guarantees the scanner is always looking at real-time data.
- Model Pre-warming: One major issue with ML Kit is the "cold start" delay. To fix this, when `QRPay.initialize()` is called, we silently push a 1x1 dummy frame through the ML pipeline on a background thread. By the time the user navigates to the scanner screen, the models are already loaded in RAM.
- Symbology Filtering: The barcode parsers are strictly configured to only look for QR\_CODE formats, skipping Aztec, PDF417, etc., which speeds up inference significantly.

## 4. Core Features Implemented

- Auto-Zoom: If the QR code bounding box occupies less than 20% of the frame, the SDK calculates the target ratio and smoothly zooms in. We added a confidence check so it doesn't aggressively zoom on false positives.
- Payment Parsing: Built-in parsers validate EMVCo (checking the CRC-16 checksum before attempting a full parse to save cycles) and UPI formats.
- Thermal Management: If the device gets hot during long sessions, the SDK degrades the framerate from 30fps to 10fps to prevent battery drain.
- Sub-2MB Size: By unbundling ML Kit and relying on Google Play Services, the added APK size sits well within our 2MB budget.

## 5. Testing & QA

We ran a comprehensive suite of tests across the codebase before this release:

- Static Analysis: ``flutter analyze`` returns 0 issues across all packages.

## QRPay SDK - Technical Handoff & Overview

- Unit Tests: All config validation and auto-zoom math passed.
- Property-based Tests: We used the `dart\_check` package to run hundreds of iterations of simulated, randomly-generated EMVCo and UPI strings to ensure our parsers don't crash on edge cases.
- UI Goldens: Verified that the PositioningOverlayPainter renders the scanning box correctly on different screen ratios.
- End-to-End: Passed integration testing on physical devices to verify the 200ms camera shutdown sequence works reliably after a successful scan.

### 6. Conclusion

Overall, the SDK is stable and ready for integration. The optimizations we put in place make the camera launch and scan almost instantly, which should provide a really solid user experience for payment flows.